Recursive versus iterative algorithms

Outline

Recursion and iteration

Comparison at a glance

Divide and conquer is what gives the gain, not recursion itself

When subproblems overlap

Space

Experiment

Guidelines

Where we were

In the previous chapters we have studied several recursive algorithms:

- chapter «Recursion»: factorial, Fibonacci numbers, recursive GCD, fast power, Tower of Hanoi;
- chapter «Recursive algorithms»: sum of an array, Merge Sort, Recursive Binary Search, maximum subarray.

We have analysed them by means of recurrences and of the Master Theorem.

The central question

In this chapter we answer a very practical question:

when is it convenient to design a recursive algorithm, and when is it not?

It depends on the decomposition

The answer does not depend on recursion itself, but on **how** the problem is decomposed into subproblems:

- if the problem is split into a few **independent** subproblems of size n/b (divide and conquer), recursion may give a large asymptotic gain;
- if the problem is only reduced from size n to size $n-1$, recursion gives no gain in time and it costs memory;
- if the subproblems **overlap**, a naive recursive algorithm may be exponentially slower than an iterative one.

Plan of the chapter

- First, the costs that every recursive call carries
- Then a comparison of recursive and iterative algorithms for the problems of the course
- The three situations above, one by one: divide and conquer, decrease by one (with the memory used by the recursion), overlapping subproblems
- Finally, the measured running times of the C++ implementations and some guidelines

Recursion and iteration

Introduction: Recursion and iteration

- In this section we see that recursion and iteration can express exactly the same algorithms
- We look at what happens in the computer when a function is called: the **call stack**, and the three hidden costs of recursion
- Finally, we transform a recursive algorithm into an iterative one with an explicit stack, and we see what changes and what does not

Same expressive power

- Every recursive algorithm can be transformed into an iterative one (that uses an **explicit stack** to store the pending calls)
- Every iterative algorithm can be written recursively (every loop can be replaced by a function that calls itself)

Recursion and iteration have the **same expressive power**: what changes from one to the other is the **number of steps** and the **amount of memory** needed to solve the problem.

The call stack

- Each time a function is called, the computer creates a **stack frame** (or *activation record*) on the **call stack**, a memory area reserved to the program
- The stack frame contains:
  - the parameters of the call;
  - the local variables of the call;
  - the return address, i.e., the point of the calling function from which the execution resumes when the call terminates.

Active calls keep their frame

- The frame is removed from the stack only when the call terminates
- Accordingly, during the execution of a recursive algorithm, all the calls that have started and not yet terminated keep their frame on the stack
- This produces three **hidden costs**, which do not appear in a loop

The hidden costs of recursion (1/2)

1. **Call overhead**: creating and removing a frame, copying the parameters and jumping to the function and back require a constant number of extra steps for each call.
   - This does not change the order of growth of the running time
   - but it makes a recursive algorithm slower than an equivalent loop by a constant factor

2. **Memory**: the memory used by the call stack is proportional to the maximum number of calls that are *simultaneously* active, i.e., to the **depth** of the recursion (the height of the recursion tree plus one, studied in detail later in this chapter).

The hidden costs of recursion (2/2)

3. **Stack overflow**: the call stack has a fixed and rather small size.
   - On Linux the default size is typically 8 MB (command `ulimit -s` $\Rightarrow$ 8192 KB), on Windows it is typically 1 MB
   - If the depth of the recursion is too large, the stack is exhausted and the program **crashes** (in C++ with a *segmentation fault*)
   - In Python the default limit is only 1000 nested calls

Example (1/2)

A recursive linear sum that crashes.

- Consider the decrease-by-one recursive algorithm that computes the sum of an array $a[1..n]$ as

$$\texttt{SumDecrease}(a, n) = \texttt{SumDecrease}(a, n-1) + a[n],$$

  with base case $\texttt{SumDecrease}(a, 0) = 0$ (the decrease-by-one sum of the chapter «Recursive algorithms», with base case $n = 0$ instead of $n = 1$)

- The calls $\texttt{SumDecrease}(a, n), \texttt{SumDecrease}(a, n-1), \ldots, \texttt{SumDecrease}(a, 0)$ are all active at the same time: the depth of the recursion is $n + 1$

Example (2/2)

- With $n = 10^7$ and a stack frame of about 32 bytes, the stack should contain about

$$10^7 \cdot 32 = 3.2 \cdot 10^8 \text{ bytes} \approx 320 \text{ MB},$$

  i.e., 40 times the default 8 MB

- The program crashes, although the array itself ($4 \cdot 10^7$ bytes $= 40$ MB) fits easily in memory and the iterative sum computes the result in about 3 milliseconds

- In the experiment at the end of the chapter we measure that, with a stack of 8 MB, the crash occurs already for $n \approx 7 \cdot 10^5$

From recursion to iteration with an explicit stack

- A recursive algorithm can always be transformed into an iterative one by managing **explicitly** a stack that contains the parameters of the calls still to be executed
- As an example, we write an iterative version of the halving recursive sum `RecursiveSum` of the chapter «Recursive algorithms»
- `RecursiveSum` computes the sum of $a[l..r]$ as the sum of $a[l..mid]$ plus the sum of $a[mid + 1..r]$

This is the pseudo-code of the algorithm `SumExplicitStack`,
which computes the sum of an array $a[1..n]$
by keeping the sub-arrays still to be summed in a stack P.

Algorithm: SumExplicitStack (1/2)

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output : The sum $a[1] + a[2] + \cdots + a[n]$

```
1 function SumExplicitStack(a, n)
2 |   s := 0
3 |   P := empty stack                    // pending sub-arrays
4 |   push (1, n) on P
5 |   while P is not empty do
6 |   |   (l, r) := pop from P
```

Algorithm: SumExplicitStack (2/2)

```
function SumExplicitStack(a, n) (continued)
    while P is not empty (continued) do
7       if l = r then
8           s ← s + a[l]                          // base case
9       else
10          mid := ⌊(l + r)/2⌋
11          push (mid + 1, r) on P
12          push (l, mid) on P
13      end
14  end
15  return s
16 end
```

Run in Colab: Python · C++ · Java

The role of the stack P

- The stack P plays exactly the role of the call stack: it contains the sub-arrays still to be summed
- Its maximum size is of the same order as the depth of the recursion, i.e.,

$$\Theta(\log_2 n)$$

Example (1/2)

We execute `SumExplicitStack`$(a, 4)$ with $a = [4, 7, 1, 9]$. Each row reports the pair popped from P, what is done, and the content of P afterwards (the top of the stack is on the right):

pop	action	P afterwards	s
$(1, 4)$	$mid = 2$, push $(3, 4)$ and $(1, 2)$	$[(3, 4), (1, 2)]$	0
$(1, 2)$	$mid = 1$, push $(2, 2)$ and $(1, 1)$	$[(3, 4), (2, 2), (1, 1)]$	0
$(1, 1)$	base case	$[(3, 4), (2, 2)]$	4
$(2, 2)$	base case	$[(3, 4)]$	11
$(3, 4)$	$mid = 3$, push $(4, 4)$ and $(3, 3)$	$[(4, 4), (3, 3)]$	11
$(3, 3)$	base case	$[(4, 4)]$	12
$(4, 4)$	base case	empty	21

Example (2/2)

- The algorithm returns

$$21 = 4 + 7 + 1 + 9$$

- The sub-arrays are processed in the same order as the calls of $\texttt{RecursiveSum}(a, 1, 4)$
- P never contains more than 3 pairs

What the explicit stack changes

- Simulating the recursion with an explicit stack does **not** change the order of growth of the number of steps and of the memory
- It only removes the call overhead and moves the stack from the (small) call stack to the (large) main memory

A real gain in time or in memory can only come from a **different algorithm**,
not from the mechanical replacement of recursion by iteration (or vice versa).

Summary

Recursion and iteration:

- same expressive power: what changes is the number of steps and the memory

Hidden costs of recursion:

- call overhead (a constant factor), memory proportional to the depth of the recursion, risk of stack overflow

Explicit stack:

- `SumExplicitStack` simulates `RecursiveSum` with a stack of $\Theta(\log_2 n)$ pairs
- same order of growth: a real gain needs a different algorithm

Comparison at a glance

Introduction: Comparison at a glance

- In this section we compare, for the problems studied in the course, the best iterative algorithm of the course with the recursive one
- The comparison gives a map of the chapter: the cases where recursion pays off, those where it only replaces a loop, and the case where it is a disaster
- Each of these cases is studied in detail in the following sections

How to read the comparison

- **Time** is the worst-case number of steps $S(n)$ (arithmetic operations on numbers are counted as one step, as in the chapter «Running time»)
- **Space** is the memory used **in addition to the input**, including the call stack
- The verdict refers to the recursive algorithm:
  - **GAIN** if it is asymptotically faster;
  - **SAME** if it has the same running time;
  - **LOSS** if it is worse in time or in space.
- $\varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$ is the golden ratio and $m = \min(a, b)$ for the GCD of a and b

Recursion pays off (1/2)

Problem	**Iterative**	**time**	**space**	**Recursive**	**time**	**space**
Sorting	Insertion, Selection, Exchange Sort	$\Theta(n^2)$	$\Theta(1)$	Merge Sort	$\Theta(n \log_2 n)$	$\Theta(n)$
Power x^n	n multiplications	$\Theta(n)$	$\Theta(1)$	Fast power	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$
Maximum subarray	Brute force	$\Theta(n^2)$	$\Theta(1)$	Divide and conquer	$\Theta(n \log_2 n)$	$\Theta(\log_2 n)$
Product of n-digit integers	Schoolbook	$\Theta(n^2)$	$\Theta(n)$	Karatsuba	$\Theta(n^{\log_2 3})$	$\Theta(n)$

Recursion pays off (2/2)

Problem	**Verdict**
Sorting	**GAIN** in time (divide and conquer), paid with $\Theta(n)$ memory
Power x^n	**GAIN**, which comes from halving n: the iterative square-and-multiply is also $\Theta(\log_2 n)$, with $\Theta(1)$ space
Maximum subarray	**GAIN** over brute force, but the iterative Kadane algorithm, $\Theta(n)$ time and $\Theta(1)$ space, beats both
Product of n-digit integers	**GAIN**: $S(n) = 3S(n/2) + \Theta(n)$, $n^{\log_2 3} \approx n^{1.585}$

Recursion only replaces a loop (1/2)

Problem	**Iterative**	**time**	**space**	**Recursive**	**time**	**space**
Array sum	Loop	$\Theta(n)$	$\Theta(1)$	Halving	$\Theta(n)$	$\Theta(\log_2 n)$
				Decr. by one	$\Theta(n)$	$\Theta(n)$
Binary search	Loop	$\Theta(\log_2 n)$	$\Theta(1)$	Recursive Binary Search	$\Theta(\log_2 n)$	$\Theta(\log_2 n)$
Factorial, linear search	Loop	$\Theta(n)$	$\Theta(1)$	Decrease by one	$\Theta(n)$	$\Theta(n)$
GCD (Euclid)	Loop	$\Theta(\log m)$	$\Theta(1)$	Recursive GCD	$\Theta(\log m)$	$\Theta(\log m)$

Recursion only replaces a loop (2/2)

Problem	**Verdict**
Array sum	**SAME** time, **LOSS** in space
Binary search	**SAME** time, **LOSS** in space; tail recursion, it can be removed
Factorial, linear search	**SAME** time, **LOSS** in space, risk of stack overflow
GCD (Euclid)	**SAME**: tail recursion, the depth is only logarithmic

Overlapping subproblems and Tower of Hanoi

Problem	**Iterative**	**time**	**space**	**Recursive**	**time**	**space**
Fibonacci numbers	Loop with two variables	$\Theta(n)$	$\Theta(1)$	Naive recursion	$\Theta(\varphi^n)$	$\Theta(n)$
Tower of Hanoi	Iterative versions exist	$\Theta(2^n)$	$\Theta(n)$	Recursive Hanoi	$\Theta(2^n)$	$\Theta(n)$

Verdict:

- Fibonacci numbers: huge **LOSS**: overlapping subproblems; memoization brings it back to $\Theta(n)$
- Tower of Hanoi: **SAME**: $2^n - 1$ moves are needed anyway; recursion is the natural description

Three blocks, three situations

- The four rows of the first block are the cases where recursion **pays off**: in all of them the problem is split into subproblems of size $n/2$ (next section)
- The rows of the second block are the cases where recursion only **replaces a loop**: no gain in time, a loss in memory (section «Space»)
- The Fibonacci row is the case where naive recursion is a **disaster**, because the same subproblems are solved over and over again (section «When subproblems overlap»)

Summary

Recursion pays off:

- sorting, power, maximum subarray, product of integers: subproblems of size $n/2$

Recursion only replaces a loop:

- array sum, binary search, factorial, linear search, GCD: same time, more memory

Recursion is a disaster:

- Fibonacci numbers with naive recursion: $\Theta(\varphi^n)$ instead of $\Theta(n)$

Divide and conquer is what gives the gain, not recursion itself

Introduction: Divide and conquer

- In this section we compare the two families of recurrences of the course: decrease by one and divide and conquer
- We show that a decrease-by-one recursion does the same work as a loop, while divide and conquer can give an asymptotic gain
- For Merge Sort and the fast power we write iterative algorithms with the same running time: the gain comes from the decomposition, not from recursion
- We also meet an iterative algorithm that beats divide and conquer (Kadane)

Two families of recurrences

The number of steps of a recursive algorithm is described by a recurrence. Two families of recurrences appear in the course:

- **decrease by one**: the problem of size n is reduced to **one** problem of size $n-1$,

$$S(n) = S(n-1) + f(n);$$

- **divide and conquer**: the problem of size n is split into a problems of size n/b,

$$S(n) = a\,S(n/b) + f(n), \qquad a \geq 1,\ b > 1.$$

Decrease by one: recursion is a loop in disguise

Observation 1: decrease-by-one recurrences

Let $S(n) = S(n-1) + f(n)$ *for* $n \geq 1$, *with* $S(0) = d$. *Then*

$$S(n) = d + \sum_{i=1}^{n} f(i).$$

In particular, if $f(n) = c$ *for a constant* $c > 0$, *then* $S(n) = c\,n + d = \Theta(n)$.

Proof

By induction on n. For $n = 0$ the sum is empty and $S(0) = d$.

If the formula holds for $n - 1$, then

$$S(n) = S(n-1) + f(n)$$

$$= d + \sum_{i=1}^{n-1} f(i) + f(n)$$

$$= d + \sum_{i=1}^{n} f(i).$$

If $f(i) = c$ for every i, then $\sum_{i=1}^{n} c = c\,n$ and $S(n) = c\,n + d$. Since

$$\lim_{n\to\infty} \frac{c\,n + d}{n} = c \in \mathbb{R}_{>0},$$

we have $S(n) = \Theta(n)$. □

A loop in disguise (1/2)

- The formula

$$S(n) = d + \sum_{i=1}^{n} f(i)$$

  is **exactly** the number of steps of a loop that executes, for $i = 1, 2, \ldots, n$, a body of cost $f(i)$

- The decrease-by-one recursion does the same work as a loop

A loop in disguise (2/2)

- Factorial, linear search and the decrease-by-one array sum:

$$S(n) = S(n-1) + \Theta(1) = \Theta(n),$$

  like the corresponding loop
- A recursive insertion sort (sort $a[1..n-1]$ recursively, then insert $a[n]$):

$$S(n) = S(n-1) + \Theta(n) = \Theta(n^2),$$

  like the iterative `InsertionSort`
- In all these cases the recursive algorithm has the same order of growth of the running time as the iterative one, plus the call overhead and a depth of recursion $\Theta(n)$

Divide and conquer: where the gain comes from

- Merge Sort splits the array into two halves, sorts them recursively and merges them in $\Theta(n)$ steps:

$$S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n)$$

  by case 2 of the Master Theorem

- The gain with respect to $\Theta(n^2)$ comes from the fact that the two halves are **balanced** and **independent**: the recursion tree has only $\Theta(\log_2 n)$ levels, and each level costs $\Theta(n)$

Bottom-up Merge Sort

- The same splitting can be organised **without recursion**:
  - we first merge pairs of sub-arrays of length 1,
  - then pairs of sub-arrays of length 2,
  - then of length 4, and so on.

- This is the **bottom-up** Merge Sort

- It uses the procedure `Merge`(a, l, m, r) of the chapter «Recursive algorithms», which merges the sorted sub-arrays $a[l..m]$ and $a[m+1..r]$ in $\Theta(r - l + 1)$ steps

This is the pseudo-code of the algorithm `MergeSortBottomUp`,
which sorts an array $a[1..n]$
by merging sub-arrays of length $1, 2, 4, \ldots$

Algorithm: MergeSortBottomUp (1/2)

Input : An array $a[1..n]$ of n natural numbers
Output : The array a sorted in non-decreasing order

```
1 function MergeSortBottomUp(a, n)
2 |   width := 1
3 |   while width < n do
4 |   |   l := 1
5 |   |   while l + width ≤ n do
6 |   |   |   m := l + width − 1
```

Algorithm: MergeSortBottomUp (2/2)

```
function MergeSortBottomUp(a, n) (continued)
    while width < n (continued) do
        while l + width ≤ n (continued) do
7           r := min(l + 2 · width − 1, n)
8           Merge(a, l, m, r)
9           l ← l + 2 · width
10      end
11      width ← 2 · width
12  end
13 end
```

Run in Colab: Python · C++ · Java

Example

Consider $a = [5, 2, 4, 6, 1, 3]$ ($n = 6$).

- $width = 1$: merge $a[1..1]$ with $a[2..2]$, $a[3..3]$ with $a[4..4]$, $a[5..5]$ with $a[6..6] \Rightarrow a = [2, 5, 4, 6, 1, 3]$
- $width = 2$: merge $a[1..2]$ with $a[3..4]$; for $l = 5$ we have $l + width = 7 > 6$, so $a[5..6]$ is left as it is $\Rightarrow$ $a = [2, 4, 5, 6, 1, 3]$
- $width = 4$: merge $a[1..4]$ with $a[5..6] \Rightarrow a = [1, 2, 3, 4, 5, 6]$
- $width = 8 \geq 6$: stop

The algorithm executes $3 = \lceil \log_2 6 \rceil$ passes.

Observation 2: bottom-up Merge Sort

For $n \geq 2$, `MergeSortBottomUp` executes $\lceil \log_2 n \rceil$ passes of the outer loop and its number of steps in the worst case is

$$S(n) = \Theta(n \log_2 n),$$

the same as the recursive Merge Sort. Moreover, it does not use the call stack.

- Approach of the proof: we bound the cost of one pass from above and from below by a multiple of n, and we count the passes

Proof (1/3)

Let $n \geq 2$.

In the outer loop *width* takes the values

$$1, 2, 4, \ldots, 2^{p-1},$$

where p is the smallest integer with $2^p \geq n$, i.e.,

$$p = \lceil \log_2 n \rceil.$$

Proof (2/3)

Upper bound.

- In a pass, the calls to `Merge` (line 8) work on disjoint sub-arrays, so their total length is at most n and their total cost is $O(n)$
- The inner loop is executed at most $\frac{n}{2 \cdot width} + 1 \leq n$ times, each time with a constant number of other steps
- Hence every pass costs $O(n)$ and

$$S(n) = O(n \lceil \log_2 n \rceil) = O(n \log_2 n).$$

Proof (3/3)

Lower bound.

- The inner loop stops when $l + width > n$, so the elements that are not merged in a pass are at most $width$
- In the passes with $width \leq n/2$, at least $n - width \geq n/2$ elements are merged, with $\Omega(n)$ steps
- These passes are those with $width = 2^t$, $t = 0, 1, \ldots, \lfloor \log_2 n \rfloor - 1$, i.e., $\lfloor \log_2 n \rfloor$ passes. Hence $S(n) = \Omega(n \lfloor \log_2 n \rfloor) = \Omega(n \log_2 n)$

Therefore $S(n) = \Theta(n \log_2 n)$. The algorithm contains no recursive call, so the only extra memory is the one used by `Merge`, i.e., $\Theta(n)$.

□

The gain comes from divide and conquer

The gain of Merge Sort over the $\Theta(n^2)$ sorting algorithms is a consequence of the **divide and conquer** strategy (balanced halves, merged in linear time), not of recursion.

The iterative bottom-up version has the same running time $\Theta(n \log_2 n)$.

Fast power: the gain comes from halving

- The recursive fast power of the chapter «Recursion» computes

$$x^n = \begin{cases} \left(x^{\lfloor n/2 \rfloor}\right)^2 & \text{if } n \text{ is even,} \\ \left(x^{\lfloor n/2 \rfloor}\right)^2 \cdot x & \text{if } n \text{ is odd,} \end{cases}$$

  with

$$S(n) = S(\lfloor n/2 \rfloor) + \Theta(1) = \Theta(\log_2 n)$$

- Halving the exponent can also be done with a loop, by reading the bits of n from the least significant one (**square-and-multiply**)

Algorithm: PowerIt (1/2)

Input : A number x and a natural number n
Output : The power x^n

```
1 function PowerIt(x, n)
2 |   p := 1
3 |   y := x
4 |   k := n
5 |   while k > 0 do
6 |   |   if k is odd then
7 |   |   |   p ← p · y
8 |   |   end
```

Algorithm: PowerIt (2/2)

```
function PowerIt(x, n) (continued)
    while k > 0 (continued) do
9       y ← y · y
10      k ← ⌊k/2⌋
11  end
12  return p
13 end
```

Run in Colab: Python · C++ · Java

Why PowerIt is correct

- At the beginning of each iteration we have

$$p \cdot y^k = x^n$$

- It holds at the start with $p = 1$, $y = x$, $k = n$, and it is preserved by every iteration

- When $k = 0$ we get $p = x^n$

Example

We execute `PowerIt`$(x, 13)$, where $13 = (1101)_2$. Each item reports the values at the end of an iteration:

- $k = 13$ is odd: $p = x$, $y = x^2$, $k = 6$;
- $k = 6$ is even: $p = x$, $y = x^4$, $k = 3$;
- $k = 3$ is odd: $p = x \cdot x^4 = x^5$, $y = x^8$, $k = 1$;
- $k = 1$ is odd: $p = x^5 \cdot x^8 = x^{13}$, $y = x^{16}$, $k = 0$.

The iterations multiply p by x, x^4 and x^8 (the bits of 13 equal to 1), and

$$p = x^{1+4+8} = x^{13}$$

after $4 = \lfloor \log_2 13 \rfloor + 1$ iterations.

Observation 3: iterations of PowerIt

For $n \geq 1$, `PowerIt` executes exactly $\lfloor \log_2 n \rfloor + 1$ iterations, hence $S(n) = \Theta(\log_2 n)$, and it uses $\Theta(1)$ extra memory.

Proof

- Every iteration replaces k by $\lfloor k/2 \rfloor$, i.e., it removes the least significant bit of the binary representation of k
- Hence the number of iterations is the number of binary digits of n, which is

$$\lfloor \log_2 n \rfloor + 1$$

(chapter «Representation of numbers»)

- Each iteration executes a constant number of steps, so $S(n) = \Theta(\log_2 n)$
- Only the three variables p, y, k are used

□

Maximum subarray: a better iterative algorithm

- For the maximum subarray problem (chapter «Recursive algorithms») the divide and conquer algorithm, with

$$S(n) = 2S(n/2) + \Theta(n) = \Theta(n \log_2 n),$$

  improves on the $\Theta(n^2)$ brute force algorithm
- However, a smarter **iterative** algorithm, due to Kadane, solves the problem in $\Theta(n)$ steps
- For each position i it maintains the maximum sum *cur* of a sub-array **ending** at position i: such a sub-array either is $a[i]$ alone, or extends the best sub-array ending at $i - 1$

This is a compact version of the algorithm `Kadane`
of the chapter «Recursive algorithms»,
which returns only the maximum sum.

Algorithm: Kadane

Input : An array $a[1..n]$ of $n \geq 1$ integer numbers
Output : The maximum sum of a non-empty sub-array $a[l..r]$

```
1  function Kadane(a, n)
2      best := a[1]
3      cur := a[1]   // best sum of a sub-array ending at i
4      i := 2
5      while i ≤ n do
6          cur ← max(a[i], cur + a[i])
7          best ← max(best, cur)
8          i ← i + 1
9      end
10     return best
11 end
```

Run in Colab: Python · C++ · Java

Kadane beats divide and conquer

- The loop is executed $n-1$ times with a constant number of steps:

$$S(n) = \Theta(n),$$

  with $\Theta(1)$ extra memory

- Divide and conquer is a powerful **design technique**, but it is not always the best one

- A gain with respect to brute force does not mean that the recursive algorithm is optimal

Integer multiplication: Karatsuba

- Two n-digit integers are multiplied with the schoolbook method in $\Theta(n^2)$ steps on the digits
- Splitting each number into two halves of $n/2$ digits, the product requires four products of $n/2$-digit numbers
- Karatsuba observed that **three** products are enough (plus additions and shifts, costing $\Theta(n)$):

$$S(n) = 3S(n/2) + \Theta(n)$$

- Here $a = 3$, $b = 2$, $f(n) = \Theta(n)$ and $n^{\log_2 3} \approx n^{1.585}$; since $f(n) = O(n^{\log_2 3 - \varepsilon})$ for $\varepsilon = 0.5$, case 1 of the Master Theorem gives $S(n) = \Theta(n^{\log_2 3})$, asymptotically better than $\Theta(n^2)$

Key points

- A decrease-by-one recursion does the same work as a loop
- A divide and conquer recursion can be asymptotically faster than the simple iterative algorithms (Merge Sort, fast power, Karatsuba)
- But the gain comes from the decomposition into balanced independent subproblems, and the same decomposition can often be organised with a loop (bottom-up Merge Sort, square-and-multiply)
- The best algorithm for a problem may even be iterative (Kadane)

Summary (1/2)

Decrease by one:

- $S(n) = S(n-1) + f(n)$ gives $S(n) = d + \sum_{i=1}^{n} f(i)$: the steps of a loop
- same running time as the loop, plus call overhead and depth $\Theta(n)$

Divide and conquer:

- balanced independent halves: Merge Sort in $\Theta(n \log_2 n)$
- `MergeSortBottomUp`: the same $\Theta(n \log_2 n)$ without recursion

Summary (2/2)

Fast power:

- `PowerIt` (square-and-multiply): $\lfloor \log_2 n \rfloor + 1$ iterations, $\Theta(1)$ memory

Maximum subarray:

- the iterative `Kadane` in $\Theta(n)$ beats divide and conquer in $\Theta(n \log_2 n)$

Karatsuba:

- $S(n) = 3S(n/2) + \Theta(n) = \Theta(n^{\log_2 3})$, better than $\Theta(n^2)$

When subproblems overlap

Introduction: When subproblems overlap

- In this section we study the case where the subproblems are not disjoint
- On the Fibonacci numbers, we see that the naive recursion executes an exponential number of calls
- Then we remove the waste in two ways: recursion with memoization, and a simple loop
- This is the idea of Dynamic Programming, which will be studied later in the course

Disjoint and overlapping subproblems

- In divide and conquer the subproblems are **disjoint**: the two halves of Merge Sort have no element in common
- When the subproblems **overlap**, a naive recursive algorithm solves the same subproblem many times
- The Fibonacci numbers are defined by

$$F(0) = 0, \qquad F(1) = 1,$$

$$F(n) = F(n-1) + F(n-2) \quad \text{for } n \geq 2$$

- The direct translation of the definition is the algorithm `FibRec`

Algorithm: FibRec

Input : A natural number n
Output : The Fibonacci number $F(n)$

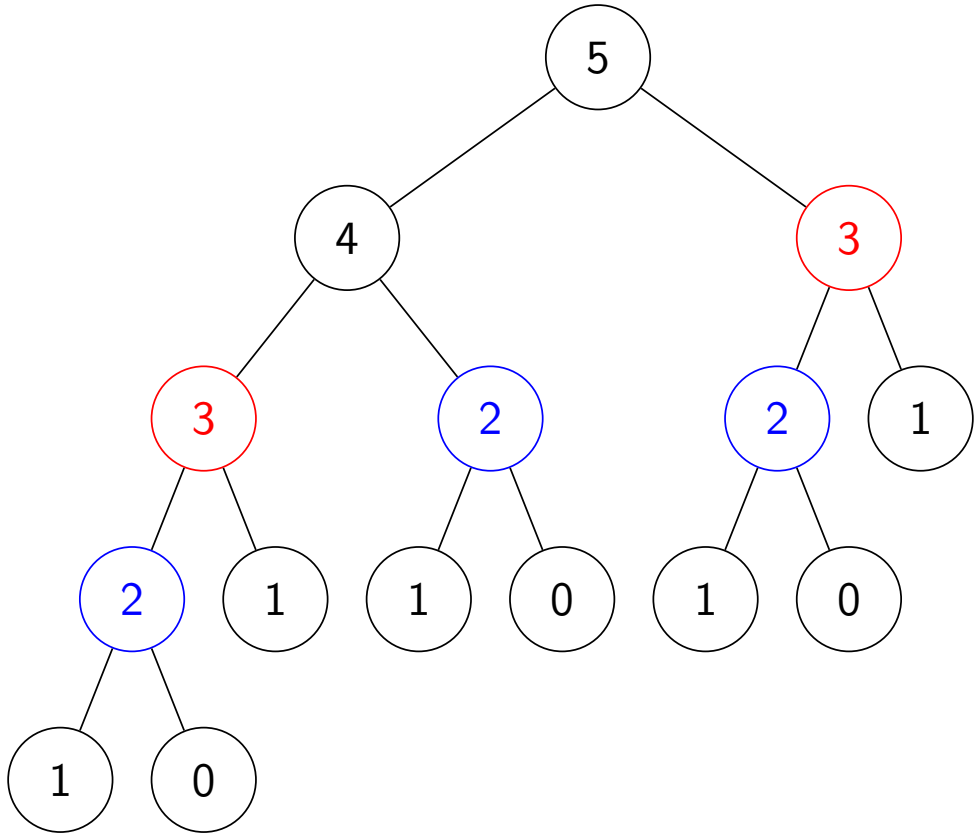
```
1 function FibRec(n)
2 |   if n ≤ 1 then
3 |   |   return n
4 |   end
5 |   return FibRec(n − 1) + FibRec(n − 2)
6 end
```

Run in Colab: Python · C++ · Java

Example (1/2)

- On the next slide: the recursion tree of `FibRec(5)`
- Each node is a call, labelled with its parameter
- The sub-tree of `FibRec(3)` is computed twice, the call `FibRec(2)` three times

Example (2/2)



The tree has 15 nodes, while only 6 different values $F(0), F(1), \ldots, F(5)$ are needed.

Observation 4: number of calls of FibRec

For every $n \geq 0$, the number of calls executed by `FibRec`(n) *is*

$$C(n) = 2F(n+1) - 1.$$

Proof

The calls satisfy

$$C(0) = C(1) = 1, \qquad C(n) = 1 + C(n-1) + C(n-2) \text{ for } n \geq 2.$$

We prove the formula by strong induction.

Base cases: $C(0) = 1 = 2F(1) - 1$ and $C(1) = 1 = 2F(2) - 1$.

If the formula holds for $n-1$ and $n-2$, then

$$\begin{aligned} C(n) &= 1 + \big(2F(n) - 1\big) + \big(2F(n-1) - 1\big) \\ &= 2\big(F(n) + F(n-1)\big) - 1 \\ &= 2F(n+1) - 1. \end{aligned}$$



Observation 5: growth of the Fibonacci numbers

Let $\varphi = \frac{1+\sqrt{5}}{2}$. *For every* $n \geq 1$,

$$\varphi^{n-2} \leq F(n) \leq \varphi^{n-1}.$$

Hence $F(n) = \Theta(\varphi^n)$ *and the number of steps of* `FibRec` *is* $S(n) = \Theta(C(n)) = \Theta(\varphi^n)$.

Proof (1/2)

The number φ satisfies $\varphi^2 = \varphi + 1$.

Base cases:

$$\varphi^{-1} \approx 0.618 \leq F(1) = 1 \leq \varphi^0 = 1,$$

$$\varphi^0 = 1 \leq F(2) = 1 \leq \varphi \approx 1.618.$$

For $n \geq 3$, by strong induction,

$$\begin{aligned} F(n) &= F(n-1) + F(n-2) \\ &\geq \varphi^{n-3} + \varphi^{n-4} = \varphi^{n-4}(\varphi + 1) = \varphi^{n-4}\varphi^2 = \varphi^{n-2}. \end{aligned}$$

Proof (2/2)

In the same way

$$F(n) \leq \varphi^{n-2} + \varphi^{n-3} = \varphi^{n-3}(\varphi + 1) = \varphi^{n-1}.$$

Accordingly,

$$\varphi^{-2}\varphi^n \leq F(n) \leq \varphi^{-1}\varphi^n, \quad \text{i.e.,} \quad F(n) = \Theta(\varphi^n).$$

Each call executes a constant number of steps (besides its sub-calls), so

$$S(n) = \Theta(C(n)) = \Theta(2F(n+1) - 1) = \Theta(\varphi^n).$$

□

How many calls?

- `FibRec(40)` executes

$$2F(41) - 1 = 331\,160\,281$$

calls to compute $F(40) = 102\,334\,155$

- `FibRec(50)` executes

$$2F(51) - 1 = 40\,730\,022\,147$$

calls

Memoization

- The waste of `FibRec` is eliminated by storing in a table $memo[0..n]$ the values already computed (**memoization**)
- Before computing $F(k)$ we check whether it is already in the table

Algorithm: FibMemo

Input : A natural number n and a table $memo[0..n]$ whose entries are all *undefined*

Output : The Fibonacci number $F(n)$

```
1 function FibMemo(n, memo)
2     if n ≤ 1 then
3         return n
4     end
5     if memo[n] is undefined then
6         memo[n] ←
            FibMemo(n − 1, memo) + FibMemo(n − 2, memo)
7     end
8     return memo[n]
9 end
```

Run in Colab: Python · C++ · Java

Observation 6: number of calls of FibMemo

For every $n \geq 2$, `FibMemo`$(n, memo)$ (with all entries of memo undefined) executes exactly $2n-1$ calls. Hence $S(n) = \Theta(n)$; the memory used is $\Theta(n)$ (the table and the call stack).

Proof (1/2)

Let $M(k)$ be the number of calls executed by `FibMemo`$(k, memo)$ when the entries $memo[0..k]$ are undefined.

- For $k = 2$ the call makes two calls, both base cases: $M(2) = 3$
- For $k \geq 3$ the call makes the call `FibMemo`$(k-1, memo)$, which executes $M(k-1)$ calls and fills $memo[2..k-1]$
- Then it makes the call `FibMemo`$(k-2, memo)$, which finds $memo[k-2]$ already defined (or is a base case if $k-2 \leq 1$) and returns immediately

Proof (2/2)

Hence

$$M(k) = 1 + M(k-1) + 1 = M(k-1) + 2,$$

and by Observation 1

$$M(n) = M(2) + 2(n-2) = 2n - 1.$$

- Each call executes a constant number of steps, so $S(n) = \Theta(n)$
- The table has $n+1$ entries and the depth of the recursion is n

□

The iterative algorithm

- Computing the values $F(2), F(3), \ldots, F(n)$ in increasing order, each value only needs the two previous ones
- We do not even need the table: two variables are enough

Algorithm: FibIt (1/2)

Input : A natural number n
Output : The Fibonacci number $F(n)$

```
1 function FibIt(n)
2 |   if n ≤ 1 then
3 |   |   return n
4 |   end
5 |   prev := 0                          // F(i − 2)
6 |   curr := 1                          // F(i − 1)
7 |   i := 2
```

Algorithm: FibIt (2/2)

```
function FibIt(n) (continued)
8     while i ≤ n do
9         next := prev + curr
10        prev ← curr
11        curr ← next
12        i ← i + 1
13    end
14    return curr
15 end
```

Run in Colab: Python · C++ · Java

Running time of FibIt

The loop is executed $n - 1$ times:

$$S(n) = \Theta(n),$$

with $\Theta(1)$ extra memory.

Three algorithms for the Fibonacci numbers

Algorithm	time	space
`FibRec` (naive recursion)	$\Theta(\varphi^n)$	$\Theta(n)$
`FibMemo` (recursion + memoization)	$\Theta(n)$	$\Theta(n)$
`FibIt` (iteration, bottom-up)	$\Theta(n)$	$\Theta(1)$

- When the subproblems overlap, the right technique is to solve each subproblem **once** and store its value: top-down with memoization, or bottom-up with a loop
- This is the idea of **Dynamic Programming**, which will be studied later in the course

Summary

Overlapping subproblems:

- `FibRec`(n) executes $C(n) = 2F(n+1) - 1$ calls: $S(n) = \Theta(\varphi^n)$
- the recursion tree of `FibRec`(5) has 15 nodes for 6 different values

Removing the waste:

- `FibMemo`: $2n - 1$ calls, $\Theta(n)$ time and $\Theta(n)$ memory
- `FibIt`: $\Theta(n)$ time and $\Theta(1)$ memory
- solve each subproblem once: the idea of Dynamic Programming

Space

Introduction: Space

- The previous sections compared numbers of steps
- We now turn to the second resource, the memory, which for a recursive algorithm is governed by the **depth of the recursion**
- We compute the depth of the recursive algorithms of the course, and we see how **tail recursion** can be removed mechanically, obtaining a loop with constant extra memory

Definition 1: depth of the recursion

*The **depth of the recursion** $D(n)$ of a recursive algorithm on an input of size parameter n is the maximum number of calls that are active at the same time during the execution. It is the number of nodes of the longest path from the root to a leaf of the recursion tree, i.e., the height of the tree plus one.*

Depth and memory

- When the calls are executed one after the other, only the calls on the path from the root to the current call are active
- Hence, if every frame has constant size, the memory used by the call stack is $\Theta(D(n))$
- The depth is **not** the number of calls: `FibRec`(n) executes $\Theta(\varphi^n)$ calls, but at most n of them are active at the same time

Example (1/2)

The call stack of the decrease-by-one sum.

- In the code of the course the decrease-by-one sum is the C++ function `sumRecDecrease`, with arrays indexed from 0:

  $\texttt{sumRecDecrease}(a, n) = \texttt{sumRecDecrease}(a, n-1) + a[n-1]$

- We execute `sumRecDecrease(a, 3)` on the array

$a[j]$	$j = 0$	$j = 1$	$j = 2$
	4	7	1

- On the next slide: the content of the call stack (the top of the stack is on the right; `s` stands for `sumRecDecrease` with the array `a` omitted)

Example (2/2)

step	call stack (active calls)	action
1	`s(3)`	$n = 3 \neq 0$: call `s(2)`, then add $a[2]$
2	`s(3) s(2)`	$n = 2 \neq 0$: call `s(1)`, then add $a[1]$
3	`s(3) s(2) s(1)`	$n = 1 \neq 0$: call `s(0)`, then add $a[0]$
4	`s(3) s(2) s(1) s(0)`	$n = 0$: return 0
5	`s(3) s(2) s(1)`	return $0 + a[0] = 4$
6	`s(3) s(2)`	return $4 + a[1] = 11$
7	`s(3)`	return $11 + a[2] = 12$

- At step 4 there are $4 = n + 1$ active calls: the depth of the recursion is $n + 1$
- The additions are executed only **after** the base case is reached, while the stack is emptied
- For comparison, the iterative `sumIt(a, 3)` uses the two variables `s` and `i` only: $s = 0 \to 4 \to 11 \to 12$

Depth of the recursive algorithms of the course

Algorithm	base case	depth $D(n)$	$D(10^6)$
Factorial, linear sum (decrease by one)	$n = 0$	$n + 1$	1 000 001
Tower of Hanoi with n disks	$n = 0$	$n + 1$	1 000 001
`FibRec`(n), $n \geq 1$	$n \leq 1$	n	1 000 000
Sum by halving, Merge Sort, Maximum Subarray	$l = r$	$\lceil \log_2 n \rceil + 1$	21
Recursive Binary Search (worst case)	$l > r$	$\lfloor \log_2 n \rfloor + 2$	21
Fast power x^n, $n \geq 1$	$n = 0$	$\lfloor \log_2 n \rfloor + 2$	21
Recursive GCD(a, b)	$b = 0$	$\Theta(\log \min(a, b))$	–

Logarithmic and linear depth

- For the halving algorithms, the largest sub-array at level t of the recursion tree has $\lceil n/2^t \rceil$ elements, and it reaches size 1 for $t = \lceil \log_2 n \rceil$
- For the fast power, the exponent goes through the values $n, \lfloor n/2 \rfloor, \ldots, 1, 0$, which are $\lfloor \log_2 n \rfloor + 2$
- A depth $\Theta(\log_2 n)$ is never a problem in practice (21 frames for $n = 10^6$, 31 for $n = 10^9$)
- A depth $\Theta(n)$ is a real risk of stack overflow for large n

Tail recursion and its elimination

Definition 2: tail recursion

*A recursive call is a **tail call** if it is the last operation executed by the function, and its result is returned as it is, without further computation. A recursive algorithm is **tail recursive** if all its recursive calls are tail calls.*

Tail recursive or not?

- The recursive GCD (return $\texttt{GCD}(b, a \bmod b)$) and the Recursive Binary Search (return the result of the search in one half) are tail recursive
- The factorial ($n \cdot \texttt{Fact}(n-1)$), the decrease-by-one sum ($\texttt{SumDecrease}(a, n-1) + a[n]$) and `FibRec` are **not** tail recursive
- After the recursive call returns there is still a multiplication or an addition to do, so the frame must be kept

A tail call is a jump

- In a tail call, the frame of the calling function is no longer needed: nothing remains to be done after the call
- Therefore the call can be replaced by an **update of the parameters** followed by a jump to the beginning of the function
- That is, by a loop

Mechanical elimination of tail recursion

1. Enclose the body of the function in a **while** loop whose condition is the negation of the base-case condition.

2. Replace every tail call $f(e_1, \ldots, e_k)$ by the **simultaneous** assignment of the new values $e_1, \ldots, e_k$ to the parameters (use temporary variables when a new value depends on a parameter that is modified).

3. Move the return of the base case after the loop.

Example: recursive GCD ⇒ iterative GCD (1/2)

Algorithm: GCDRec

```
1 function GCDRec(a, b)
2     if b = 0 then
3         return a
4     end
5     return GCDRec(b, a mod b)
6 end
```

Algorithm: GCDIt

```
1 function GCDIt(a, b)
2     while b ≠ 0 do
3         r := a mod b
4         a ← b
5         b ← r
6     end
7     return a
8 end
```

Run in Colab: Python · C++ · Java

Example: recursive GCD ⇒ iterative GCD (2/2)

- Base condition $b = 0 \Rightarrow$ loop condition $b \neq 0$
- Tail call $\texttt{GCDRec}(b, a \bmod b) \Rightarrow (a, b) \leftarrow (b, a \bmod b)$; the temporary variable r is needed because $a \bmod b$ must be computed with the *old* value of a
- With $(a, b) = (252, 105)$ both algorithms go through the pairs

$$(252, 105), \quad (105, 42), \quad (42, 21), \quad (21, 0)$$

and return 21: four calls versus three iterations

Example: Recursive Binary Search ⇒ iterative (1/3)

Algorithm: RecBinarySearch

```
1  function RecBS(a, l, r, m)
2      if l > r then
3          return −1
4      end
5      mid := l + ⌊(r − l)/2⌋
6      if a[mid] = m then
7          return mid
8      end
9      if a[mid] < m then
10         return RecBS(a, mid + 1, r, m)
11     else
12         return RecBS(a, l, mid − 1, m)
13     end
14 end
```

Run in Colab: Python · C++ · Java

Example: Recursive Binary Search ⇒ iterative (2/3)

Algorithm: ItBinarySearch

```
1  function ItBS(a, l, r, m)
2      while l ≤ r do
3          mid := l + ⌊(r − l)/2⌋
4          if a[mid] = m then
5              return mid
6          end
7          if a[mid] < m then
8              l ← mid + 1
9          else
10             r ← mid − 1
11         end
12     end
13     return −1
14 end
```

Run in Colab: Python · C++ · Java

Example: Recursive Binary Search $\Rightarrow$ iterative (3/3)

- Base condition $l > r \Rightarrow$ loop condition $l \leq r$
- The second base case ($a[mid] = m$) stays inside the loop as a **return**
- Tail call $\texttt{RecBS}(a, mid + 1, r, m) \Rightarrow l \leftarrow mid + 1$
- Tail call $\texttt{RecBS}(a, l, mid - 1, m) \Rightarrow r \leftarrow mid - 1$ (the parameters that do not change are not assigned)
- Calling $\texttt{ItBS}(a, 1, n, m)$ we obtain exactly the iterative `BinarySearch` of the chapter «Linear and binary search»

Observation 7: elimination of tail recursion

Let f be a tail recursive algorithm whose calls execute a constant number of steps (besides the recursive call) and use frames of constant size, and let g be the iterative algorithm obtained by the mechanical elimination of the tail recursion. Then g returns the same result as f, its number of steps is of the same order as that of f, and it uses $\Theta(1)$ extra memory instead of $\Theta(D(n))$.

Proof (1/2)

- Let $p_1, p_2, \ldots, p_k$ be the values of the parameters in the successive calls of f, where p_k satisfies a base condition
- The j-th iteration of the loop of g starts with the parameters equal to p_j and executes the same operations as the call of f with parameters p_j, except that the tail call is replaced by the assignment of p_{j+1}
- Hence g goes through the same values $p_1, \ldots, p_k$ and returns, in the end, what the call with parameters p_k returns
- Since every call of f returns as it is the value of the following one, this is also the result of f

Proof (2/2)

- The k calls of f and the $k-1$ iterations of g (plus the final test and return) execute the same steps up to a constant per call
- So the two numbers of steps are of the same order
- Finally, g only uses the parameters and a constant number of temporary variables

□

Accumulators

- A function that is not tail recursive can sometimes be made tail recursive with an extra parameter (an **accumulator**) that carries the partial result
- For example, $\texttt{FactAcc}(n, acc)$ returns
  - acc if $n = 0$,
  - $\texttt{FactAcc}(n - 1, n \cdot acc)$ otherwise,

  and $n! = \texttt{FactAcc}(n, 1)$
- Its elimination gives the usual loop $acc \leftarrow acc \cdot n$, $n \leftarrow n - 1$

Do compilers do it for us?

- Optimizing compilers often eliminate tail calls automatically
- For instance, g++ 15.2 with the option `-O2` removes the recursive calls of the C++ functions `gcdRec` and `binarySearchRec` of the course code, while with `-O0` it keeps them
- However, the C++ standard does not guarantee it, and Python and Java never do it
- To be sure that the memory is $\Theta(1)$, write the loop

Key points on space

- The call stack uses memory proportional to the depth of the recursion, not to the number of calls
- A logarithmic depth is harmless; a linear depth (decrease by one) risks a stack overflow
- Tail recursion can be removed mechanically, obtaining a loop with $\Theta(1)$ extra memory

Summary

Depth of the recursion:

- maximum number of simultaneously active calls; stack memory $\Theta(D(n))$
- $\Theta(\log_2 n)$ for halving (harmless), $\Theta(n)$ for decrease by one (stack overflow)

Tail recursion:

- the recursive call is the last operation and its result is returned as it is
- mechanical elimination: `GCDRec` $\Rightarrow$ `GCDIt`, `RecBS` $\Rightarrow$ `ItBS`, with $\Theta(1)$ extra memory
- accumulators; compilers may do it, but it is not guaranteed

Experiment

Introduction: Experiment

- The asymptotic analysis predicts how the number of steps grows with n
- In this section we check these predictions on a real computer, with the C++ code of the course
- Three experiments: Insertion Sort versus Merge Sort, naive recursion versus iteration for the Fibonacci numbers, and the three versions of the array sum (time and stack)

The code of the course

- All the algorithms of this chapter are implemented in the course code, in the file `Recursion.cpp` (with `Recursion.h`); arrays are indexed from 0
- The experiments are in the file `Experiments.cpp`, and the sorting algorithms `insertionSort` and `mergeSort` are in the file `Sortings.cpp`
- Three implementations of the array sum:
  - `sumIt`: a loop;
  - `sumRecHalving`: sum by halving, depth $\Theta(\log n)$;
  - `sumRecDecrease`: sum by decrease by one, depth $\Theta(n)$.
- Three implementations of the Fibonacci numbers: `fibonacciRec`, `fibonacciMemo` (with a table of $n+1$ entries, all initialised to -1) and `fibonacciIt`

Experimental setting (1/2)

- The running times were measured on 11 October 2026 on a computer with an Intel Core i5-14500T processor (20 logical CPUs, maximum frequency 4.8 GHz), with the operating system Ubuntu 26.04 LTS (Linux kernel 7.0)
- The `C++` code of the course was compiled with g++ 15.2.0 and the option `-O2`
- The default stack size is 8 MB (`ulimit -s` $\Rightarrow$ 8192 KB)
- The function `runRecursionExperiments` of the file `Experiments.cpp` measures the wall-clock time of each computation and repeats it until at least 0.2 seconds (and at least 3 repetitions) have elapsed; each time in the tables is the average over the repetitions

Experimental setting (2/2)

- **Sorting**: arrays of $n = 10^3$, 10^4 and 10^5 random integers between 1 and 10^6 (seed 12345), sorted by `insertionSort`, by the recursive `mergeSort` and by `mergeSortBottomUp`
- **Fibonacci**: `fibonacciRec`(n) and `fibonacciIt`(n) for $n = 20, 25, 30, 35, 40$; the calls of `fibonacciRec` are counted by a counter in the code; `fibonacciIt` is repeated in blocks of 10^5 calls
- **Array sum**: `sumIt`, `sumRecHalving` and `sumRecDecrease` on arrays of $n = 10^5$, 10^6 and 10^7 random integers between 1 and 100, with the stack limit removed (`ulimit -s unlimited`); a separate run checks whether `sumRecDecrease` terminates with the default stack of 8 MB

How to read measured times

Measured times depend on the machine, on the compiler and on its options, on the other programs running, and on the cache memory.

They must not be compared in absolute terms: what matters is **how they grow** with n, which is predicted by the asymptotic analysis and is the same on every machine.

Sorting: Insertion Sort versus Merge Sort

n	`insertionSort`	`mergeSort` (recursive)	`mergeSortBottomUp`
10^3	0.0695 ms	0.0361 ms	0.0332 ms
10^4	5.38 ms	0.654 ms	0.629 ms
10^5	525 ms	7.76 ms	7.43 ms

- From $n = 10^4$ to $n = 10^5$ the time of Insertion Sort is multiplied by about 98, close to the factor 100 predicted by $\Theta(n^2)$

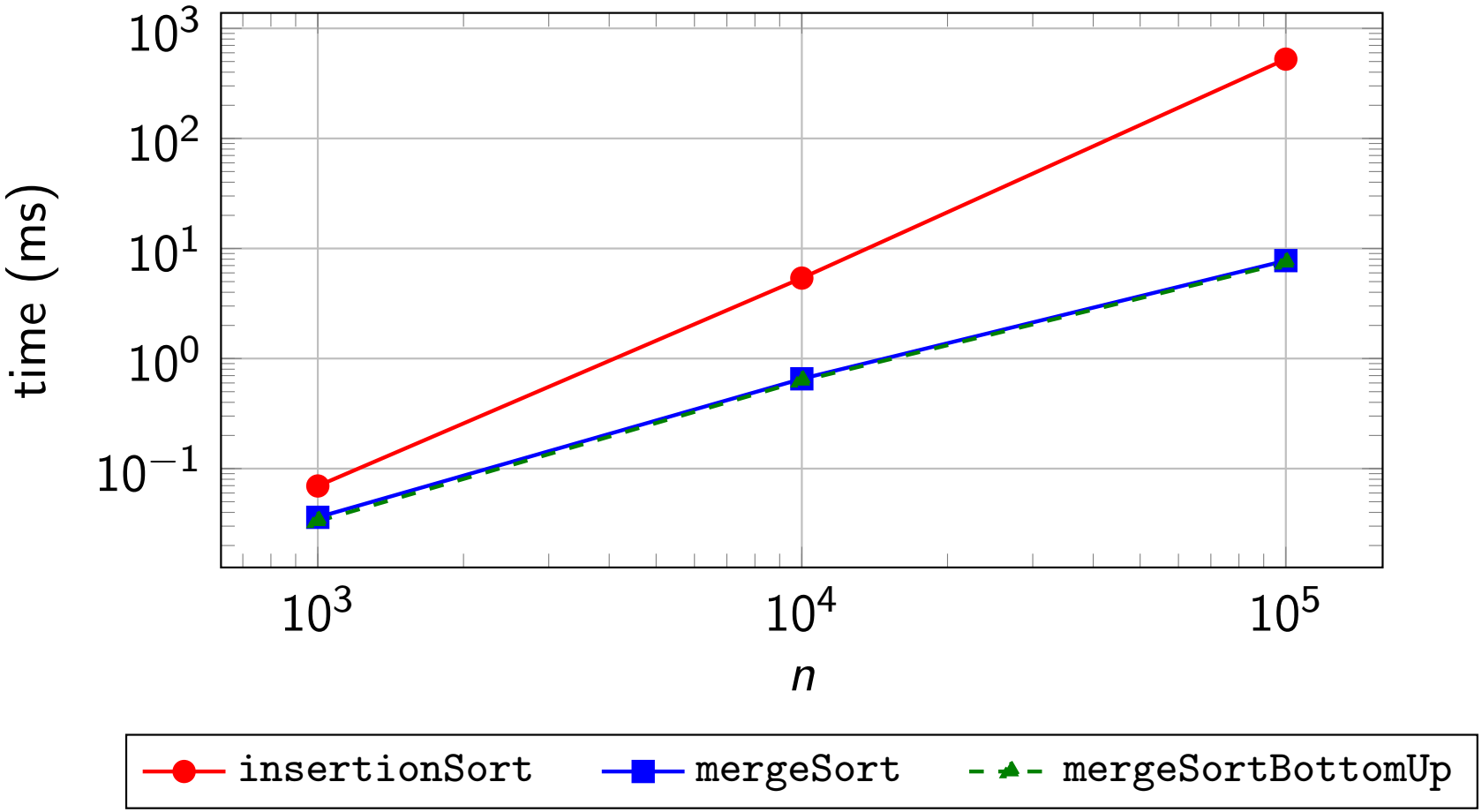
Sorting: the growth factors

- From $n = 10^4$ to $n = 10^5$ the time of Merge Sort is multiplied by about 12, close to the factor predicted by $\Theta(n \log_2 n)$:

$$10 \cdot \frac{\log_2 10^5}{\log_2 10^4} = 12.5$$

- From $n = 10^3$ to $n = 10^4$ the factors are smaller, about 77 and 18: for small arrays the constant costs and the cache memory still weigh on the measure
- For $n = 10^5$ Merge Sort is about 70 times faster
- The recursive and the bottom-up versions have practically the same time: the gain is due to divide and conquer, not to recursion

Sorting: measured running times



Logarithmic scales on both axes.

Reading a log-log plot

In a plot with logarithmic scales on both axes
a power n^k is a straight line with slope k.

The line of Insertion Sort has slope close to 2,
the lines of Merge Sort slope slightly larger than 1.

Fibonacci: naive recursion versus iteration

n	$F(n)$	calls of `fibonacciRec`	`fibonacciRec`	`fibonacciIt`
20	6 765	21 891	0.00808 ms	4.0 ns
25	75 025	242 785	0.0699 ms	4.8 ns
30	832 040	2 692 537	0.764 ms	7.4 ns
35	9 227 465	29 860 703	9.15 ms	8.4 ns
40	102 334 155	331 160 281	93.8 ms	9.5 ns

- The numbers of calls were counted with a counter in the code and coincide with $2F(n+1)-1$ (Observation 4)

Fibonacci: the growth factors

- When n increases by 5, the time of `fibonacciRec` is multiplied by a factor between about 9 and 12, in agreement with $\varphi^5 \approx 11.09$
- The time of `fibonacciIt` remains of a few nanoseconds
- Extrapolating with the factor φ per unit of n from the measured time for $n = 40$:
  - `fibonacciRec`(50) needs about $93.8 \cdot \varphi^{10} \approx 11\,500$ ms, i.e., about 12 seconds;
  - `fibonacciRec`(60) needs about $93.8 \cdot \varphi^{20}$ ms, i.e., about 24 minutes;
  - `fibonacciRec`(100) needs about $93.8 \cdot \varphi^{60}$ ms, i.e., about 10 000 years.

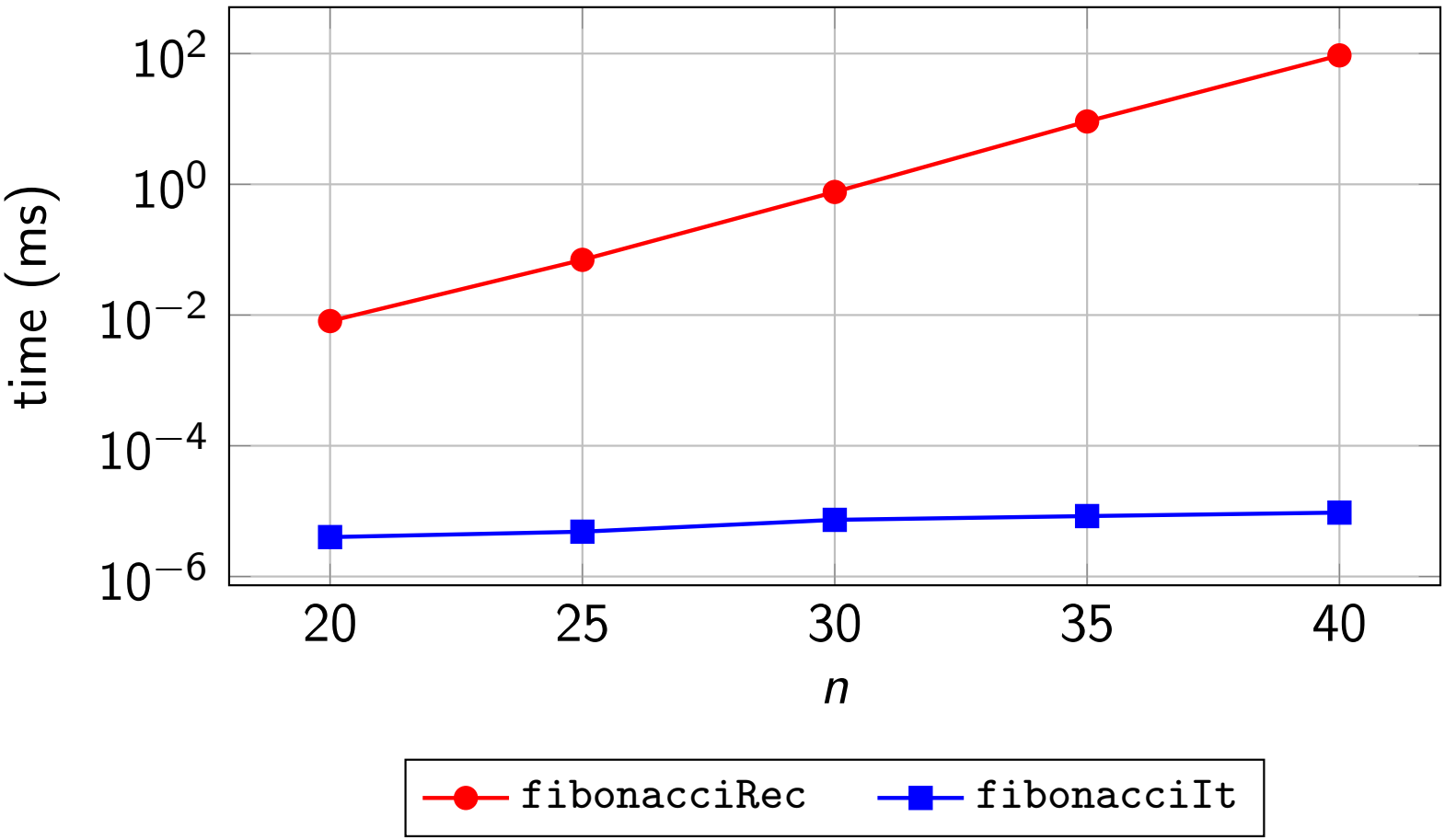
Fibonacci: the time per call

- The compiler optimizes the code: with -O2 it merges several levels of the recursion into a single function call
- So the time per call,

$$93.8 \text{ ms}/331\,160\,281 \approx 0.28 \text{ ns},$$

  is smaller than one would expect
- This changes the constant, not the exponential growth

Fibonacci: measured running times



Logarithmic scale on the vertical axis.

Reading a semi-logarithmic plot

In a semi-logarithmic plot
an exponential function is a straight line.

The slope of the red line is about

$$\log_{10} \varphi \approx 0.21$$

per unit of n.

Array sum: time and stack

n	`sumIt`	`sumRecHalving`	`sumRecDecrease`	`sumRecDecrease`, stack 8 MB
10^5	0.0215 ms	0.0960 ms	0.207 ms	OK
10^6	0.210 ms	0.910 ms	2.24 ms	crash
10^7	2.69 ms	9.01 ms	34.9 ms	crash

- The three algorithms have the same order of growth $\Theta(n)$: when n is multiplied by 10, all the times are multiplied by about 10 (between 9.5 and 16; from $n = 10^6$ to $n = 10^7$ the array no longer fits in the cache memory and the factors increase)

Array sum: call overhead and crashes

- The recursive versions are slower by a constant factor: this is the call overhead
  - about 3–4.5 for the halving version;
  - about 10–13 for the decrease-by-one version.
- With the default stack of 8 MB, `sumRecDecrease` crashes (segmentation fault) for $n = 10^6$ and $n = 10^7$
- The times of the table were measured by removing the limit with the command `ulimit -s unlimited`

Array sum: where the crash occurs (1/2)

- By bisection on n (with a precision of 10^3) we found that, with 8 MB, the largest n that does not crash is
  - about $6.97 \cdot 10^5$ with `-O2`,
  - about $2.62 \cdot 10^5$ without optimization (`-O0`).

- Without optimization every call uses a frame of 32 bytes:

$$\frac{8 \cdot 2^{20}}{32} = 262\,144$$

Array sum: where the crash occurs (2/2)

- With `-O2` the compiler merges four levels of the recursion into a single call with a frame of 48 bytes, so that

$$\frac{8 \cdot 2^{20}}{48} \cdot 4 \approx 6.99 \cdot 10^5$$

- Optimization changes the constant, not the $\Theta(n)$ depth
- `sumRecHalving` uses only $\lceil \log_2 n \rceil + 1 \leq 25$ frames for $n \leq 10^7$ and never crashes
- `sumIt` uses no frame at all

Summary

Sorting:

- factors close to 100 ($\Theta(n^2)$) and 12.5 ($\Theta(n \log_2 n)$); recursive and bottom-up Merge Sort have the same time

Fibonacci:

- a factor about $\varphi^5 \approx 11.09$ every 5 units of n for `fibonacciRec`, a few nanoseconds for `fibonacciIt`

Array sum:

- same order $\Theta(n)$, recursion slower by a constant factor
- with 8 MB of stack, `sumRecDecrease` crashes above about $7 \cdot 10^5$

Guidelines

Introduction: Guidelines

- In this last section we collect what we have learned in a few practical rules
- When to use recursion, when to avoid it, and what to check in any case
- They summarise the whole chapter

Use recursion when

- the problem splits into a few **independent** subproblems of size n/b (divide and conquer), so that the Master Theorem gives a better running time than the iterative algorithms (Merge Sort, fast power, Karatsuba);
- the recursive description is the natural one and no asymptotic gain is possible anyway (Tower of Hanoi, and later visits of trees and graphs);
- the depth of the recursion is small, typically $\Theta(\log_2 n)$.

Avoid recursion when

- the problem only decreases from n to $n-1$ with constant extra work: the recursion is a loop in disguise, with the same running time, a call overhead, $\Theta(n)$ stack memory and a risk of stack overflow (factorial, linear sum, linear search);

- the subproblems **overlap**: the naive recursion repeats the same computations and can be exponential (Fibonacci); use memoization or, better, a bottom-up loop (Dynamic Programming).

In any case

- **Always consider the space**: the call stack uses memory proportional to the depth of the recursion. Tail recursion can be eliminated mechanically, obtaining a loop with $\Theta(1)$ extra memory.
- **Always compare with the best iterative algorithm**: a recursive algorithm can improve on brute force and still be beaten by a smarter loop (Kadane's algorithm for the maximum subarray).

Summary (1/2)

Recursion and iteration:

- same expressive power; recursion pays call overhead, stack memory and risk of stack overflow

Divide and conquer:

- the gain comes from balanced independent subproblems, not from recursion (`MergeSortBottomUp`, `PowerIt`); the best algorithm may be iterative (`Kadane`)

Decrease by one:

- a loop in disguise: same time, depth $\Theta(n)$

Summary (2/2)

Overlapping subproblems:

- `FibRec` is $\Theta(\varphi^n)$; `FibMemo` and `FibIt` are $\Theta(n)$

Space:

- stack memory proportional to the depth; tail recursion becomes a loop with $\Theta(1)$ memory

Experiment:

- the measured times grow as predicted; `sumRecDecrease` crashes with the default stack